

Taller WAAP con IA/ML, RASP y DevSecOps

Informe de laboratorio - Fases 0 a 7 (v2.0)

Universidad Distrital FJDC - Mecanismos de Seguridad Informatica. Prof.
Octavio J. Salcedo Parra.

Resumen ejecutivo

Se implemento un prototipo WAAP completo: Juice Shop 20.2.0 tras ModSecurity CRS 3.3.10 (PL1), detector de anomalias IsolationForest, RASP real sobre fork de Juice Shop y pipeline DevSecOps en verde. Hallazgos centrales: (1) el CRS frena el ataque automatizado pero deja pasar payloads fragmentados; (2) el ML detecta variantes a cambio de falsos positivos (no sirve como gate duro); (3) el RASP captura con contexto de ejecucion lo que evade firmas, con overhead despreciable (+0.1 micro-s/req). La mejor cobertura es CRS+RASP con ML como senal de apoyo. Este documento (v2.0) clarifica el informe literal v1.0 y responde el cuestionario final con datos del laboratorio.

Fase 0 - Preparacion del entorno

Objetivo

Levantar la infraestructura base del laboratorio (PDF p.9).

Que se hizo (paso a paso)

```
cd taller-waap
cp .env.example .env # JUICE_PORT=3000, WAAP_PORT=8080, PARANOIA=1
docker compose pull && docker compose up -d && docker compose ps
curl -i http://localhost:8080 # 200 OK, Server: nginx (proxy WAAP)
```

Resultado y lectura

Contenedores Up (waap-proxy healthy). Nota operativa: hizo falta anadir el usuario al grupo docker (permiso denegado en /var/run/docker.sock). En Fase 4 el backend paso a la imagen local juice-shop:rasp-lab. Evidencia: Fig. 1 y Anexo B.

```
===== docker compose ps =====
NAME                                IMAGE                                COMMAND
SERVICE    CREATED        STATUS        PORTS
taller-waap-juice-shop-1    bkimminich/juice-shop:latest    "/nodejs/bin/node /jâ€¦"
juice-shop    32 minutes ago    Up 32 minutes    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1    owasp/modsecurity-crs:nginx    "/docker-entrypoint.â€¦"
waap-proxy    32 minutes ago    Up 32 minutes (healthy)    0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp

===== curl cÃ³digos =====
```

HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200

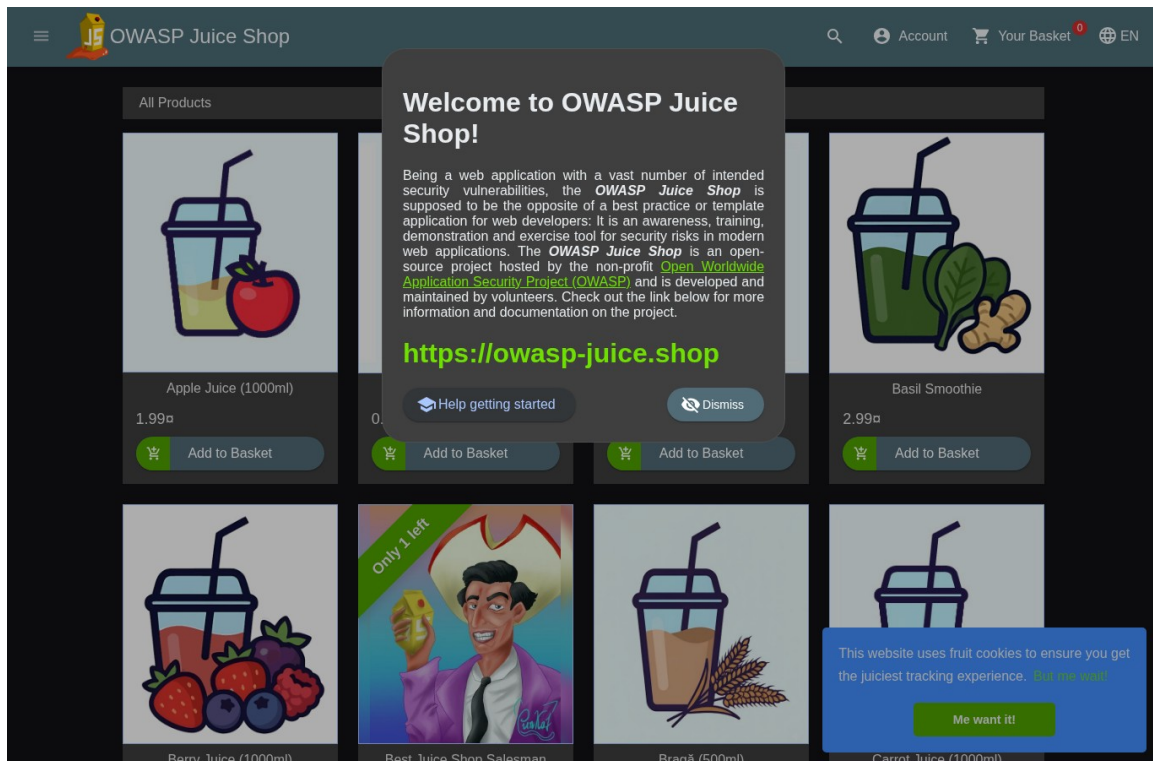
===== WAAP env =====

PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=Off
BLOCKING_PARANOIA=1

===== regla CRS (bloqueo) =====

```
34 "ruleId":"949110"  
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"  
33 "message":"Request content type is not allowed by policy"  
1 "ruleId":"942100"  
1 "message":"SQL Injection Attack Detected via libinjection"
```

Figura 1. Juice Shop servido a traves del proxy WAAP (:8080).



Fase 1 - Aplicacion objetivo vulnerable

Objetivo

Contar con un blanco controlado con vulnerabilidades documentadas (PDF p.10).

Que se hizo (paso a paso)

```
curl -s http://localhost:8080/rest/admin/application-version # version 20.2.0
curl -s 'http://localhost:8080/rest/products/search?q=apple' # JSON success via
WAAP
Navegador: http://localhost:8080/#/score-board (retos por categoria OWASP)
```

Resultado y lectura

Juice Shop 20.2.0 operativo tras el WAAP. Linea base de 5 vulnerabilidades (Tabla 1) usada en todas las fases. Evidencias: Fig. 2 (Score Board) y Fig. 3 (trafico normal).

Tabla 1. Linea base de vulnerabilidades (entregable Fase 1).

#	Vulnerabilidad	OWASP Top 10	Endpoint
1	Inyeccion SQL en login	A03 Injection	POST /rest/user/login
2	XSS reflejado en busqueda	A03 Injection	GET /rest/products/search?q=
3	IDOR en cesta/pedidos	A01 Broken Access Control	/rest/basket/:id
4	Exposicion de hash de password	A07 / Sensitive Data	GET /rest/user/whoami, /api/Users
5	Path traversal / LFI	A01 / A03	GET ...q=../../etc/passwd

Figura 2. Score Board (categorias OWASP).

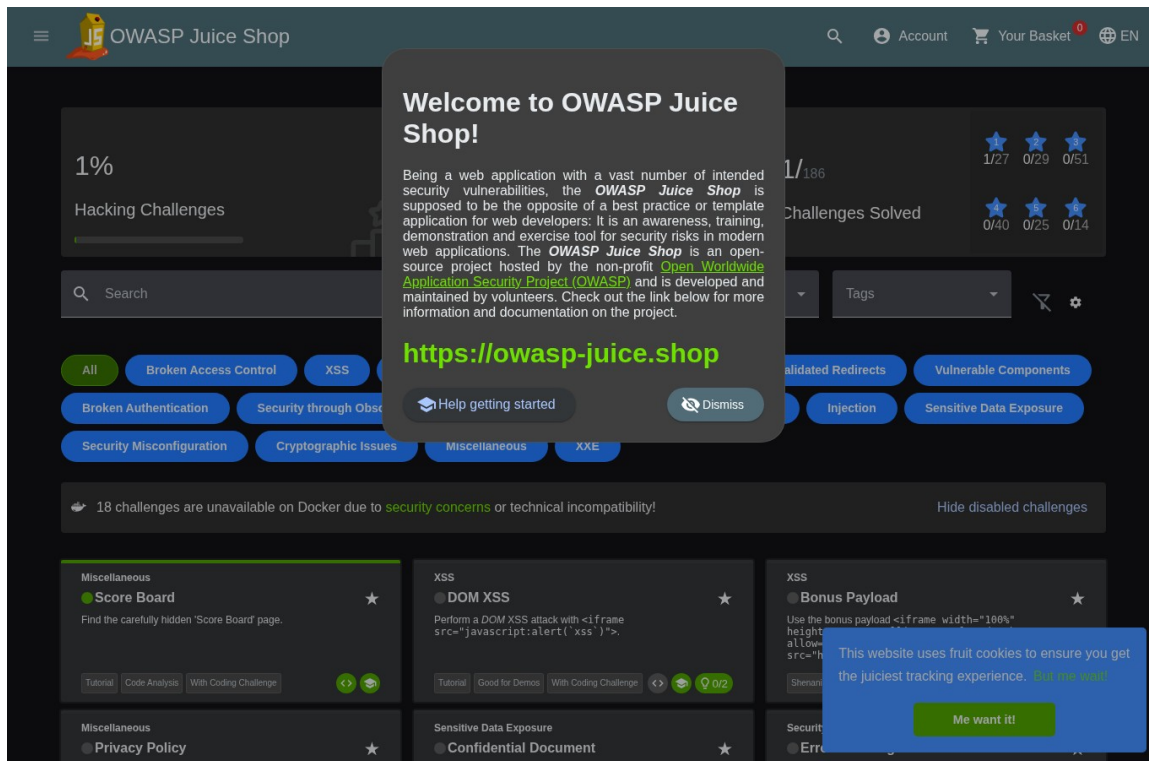


Figura 3. Trafico normal via WAAP (200).

```
Pretty-print ☐
{"status":"success","data":[{"id":1,"name":"Apple Juice (1000ml)","description":"The all-time classic.", "price":1.99, "deluxePrice":0.99, "image":"apple_juice.jpg", "createdAt":"2026-09-17 23:55:44.453 +00:00", "updatedAt":"2026-09-17 23:55:44.453 +00:00", "deletedAt":null}, {"id":24,"name":"Apple Pomace", "description":"Finest pressings of apples. Allergy disclaimer: Might contain traces of worms. Can be <a href=\"/#recycle\">sent back to us</a> for recycling.", "price":0.89, "deluxePrice":0.89, "image":"apple_pressings.jpg", "createdAt":"2026-09-17 23:55:44.462 +00:00", "updatedAt":"2026-09-17 23:55:44.462 +00:00", "deletedAt":null}, {"id":47,"name":"Pineapple Juice (1000ml)", "description":"Tropical refreshment from the finest sun-ripened pineapples.", "price":2.99, "deluxePrice":2.99, "image":"pineapple_juice.png", "createdAt":"2026-09-17 23:55:44.464 +00:00", "updatedAt":"2026-09-17 23:55:44.464 +00:00", "deletedAt":null}]}
```

Fase 2 - Capa base WAF/WAAP con CRS (PL1)

Objetivo

Primera linea de defensa con OWASP CRS: un bloqueo y una evasión evidenciados (PDF p.10).

Que se hizo (paso a paso)

```
# Paranoia PL1: PARANOIA=1, BLOCKING_PARANOIA=1, BACKEND=http://juice-shop:3000
curl -i --get --data-urlencode "q=' OR '1'='1" \
  http://localhost:8080/rest/products/search # -> 403 Forbidden
curl -i "http://localhost:8080/rest/products/search?q='%20&x=OR%201=1" # -> 200
docker compose logs waap-proxy | grep -E '942100|949110' # (el audit log sale por
docker logs)
```

Resultado y lectura

SQLi clasico/encoded/comentarios/UNION/XSS/traversal -> 403 (regla 942100 libinjection + 949110). Solo el fragmentado q=' & x=OR 1=1 -> 200: el CRS evalua cada parametro por separado. Evidencias: Fig. 4 (bloqueo) y Fig. 5 (evasión, base de Fase 3).

Figura 4. Bloqueo CRS: SQLi clasico -> 403 (nginx).

403 Forbidden

nginx

Figura 5. Evasión fragmentada -> 200 (pasa el CRS).

Pretty-print ☐

```
{"status": "success", "data": {}}
```

Fase 3 - Modulo de deteccion basado en IA/ML

Objetivo

Modelo de anomalias entrenado solo con trafico normal, capaz de senalar variantes que evadieron firmas (PDF p.11-12).

Que se hizo (paso a paso)

```
python3 -m pip install --user scikit-learn pandas numpy joblib
GEN_N=120 PACE=humano python3 waap/ml/gen_traffic.py # 120 reqs, ~15 min via WAAP
python3 waap/ml/extract_features.py # 6 features literales del PDF
python3 waap/ml/train_model.py # IsolationForest(200, 0.02) -> model.pkl
python3 waap/ml/evaluate.py # matriz 10 + 2 umbrales + rafaga bot
```

Resultado y lectura

Leccion metodologica: el primer dataset a ritmo maquina contamino req_per_minute; se regenero a ritmo humano (rpm 8+-1). Umbral -0.05: 2/5 ataques, 2 FP. Umbral 0.0: 5/5 ataques, 4 FP. La rafaga no cruza el umbral (limitacion de IsolationForest verificada). Conclusion: el ML es senal de apoyo, no gate. Detalles y numeros abajo.

Evidencias

Entrenamiento:

Modelo entrenado y guardado en waap/ml/model.pkl (120 muestras)

Matriz de 10 peticiones:

etiqueta	score	<-0.05	<0.0	esperado
busqueda comun	0.0287	normal	normal	normal
login valido	-0.1024	ANOMALO	ANOMALO	normal
home	-0.1112	ANOMALO	ANOMALO	normal
producto	-0.0141	normal	ANOMALO	normal
score-board	-0.0202	normal	ANOMALO	normal
SQLi clasico	-0.0253	normal	ANOMALO	ataque
SQLi encoded (Fase 2)	-0.0845	ANOMALO	ANOMALO	ataque
SQLi comentarios	-0.0428	normal	ANOMALO	ataque
evasion fragmentada (Fase 2)	-0.0095	normal	ANOMALO	ataque
XSS reflejado	-0.0759	ANOMALO	ANOMALO	ataque

```

--- Rafaga bot: 20 consultas rapidas (req_per_minute alto) ---
req_per_minute= 5: score==+0.1571  anomalo(<-0.05)=False
req_per_minute= 30: score==+0.1690  anomalo(<-0.05)=False
req_per_minute= 120: score==+0.1690  anomalo(<-0.05)=False

```

Umbrales:

```

umbral -0.05: TP=2/5 FN=3/5 FP=2/5 TN=3/5  deteccion=40% FPR=40%
umbral +0.00: TP=5/5 FN=0/5 FP=4/5 TN=1/5  deteccion=100% FPR=80%

```

Fase 4 - Integracion RASP sobre fork de Juice Shop

Objetivo

Proteccion en tiempo de ejecucion con contexto real: interceptar la operacion sensible aunque el payload haya superado el perimetro (PDF p.13).

Que se hizo (paso a paso)

```

gh repo fork juice-shop/juice-shop; clone --depth 1 en ~/juice-shop-rasp (rama rasp-
lab)
sudo npm i -g npm@11 (npm 10 falla con bug arborist); npm install + build
frontend/server
Nuevo lib/raspGuard.ts (mismo patron del PDF) + app.use() en server.ts
Guard sobre la query cruda real de routes/login.ts:34 -> 403 antes de Sequelize
PORT=3001 RASP_ENABLED=1 npm start; docker build -t juice-shop:rasp-lab (compose)

```

Resultado y lectura

Sin RASP el SQLi clasico devuelve el JWT del administrador (takeover total); con RASP, 403 + log antes de ejecutar. Tambien cae UNION y fragmentado. Hueco honesto: la forma string ' OR '1'='1 evade el regex literal (fix propuesto). Overhead: +0.1 micro-s/req (despreciable). Prototipo previo conservado en app/rasp-demo/; referencia Flask en app/rasp_agent.py.

Evidencias

Bateria con/sin RASP:

```

===== legit jim =====
oficial:3000 200
fork+RASP:3001 200
===== SQLi clasico email=' OR 1=1-- =====
oficial:3000 {"authentication":

```

```

{"token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoxLCJlc2VybmFtZSI6IiIsImVtYWlsIjoiYWRTaW5AanVpY2Utc2gub3AiLCJwYXNzd29yZCI6IjAxOTIwMjNhN2JiZDczMjUwNTE2ZjA2OWRmMThiNTA",
fork+RASP:3001 403
===== UNION =====
oficial:3000 200
fork+RASP:3001 403
===== search SQLi (middleware global) =====
oficial:3000 200
fork+RASP:3001 200
===== unit guard login (isSqlInjection, compilado) =====
PASS x3: legitimo=false, OR 1=1=true, UNION=true

```

Stack combinado WAAP+RASP:

```

===== stack combinado WAAP(CRS) + fork RASP (compose con juice-shop:rasp-lab) =====
NAME                IMAGE                COMMAND
SERVICE            CREATED              STATUS               PORTS
taller-waap-juice-shop-1  juice-shop:rasp-lab  "/nodejs/bin/node /jâ{"
juice-shop            About a minute ago   Up About a minute    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1  owasp/modsecurity-crs:nginx  "/docker-entrypoint.â{"
waap-proxy            23 hours ago         Up 19 minutes (healthy)  0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp
WAAP home :8080 -> 200
legit login :3000 -> 200
SQLi login :3000 -> 403 (RASP)
SQLi search :8080 -> 403 (CRS)

```

Fase 5 - Pipeline DevSecOps

Objetivo

Gate CI/CD con SAST, SCA, contenedor, IaC y DAST; despliegue solo en verde (PDF p.14).

Que se hizo (paso a paso)

Workflow: Semgrep p/owasp-top-ten; pip-audit (waap/ml/requirements.txt, nuevo); docker build app/; Trivy CRITICAL,HIGH exit 1; Checkov; ZAP baseline; deploy si success.
Replicado local + ejercicio: eval() inyectado -> rojo en SAST -> revert -> verde.

Resultado y lectura

El gate trabajo de verdad: Dockerfile sin USER, workflow sin permissions, 67 CVE en base Debian y 11 en el npm de la imagen -> Alpine + multistage + sin npm en runtime = 0 findings en las 4 herramientas.
deploy_waap_rules.sh: OK (200/403). Evidencias: evidencias/fase5/.

Fase 6 - Pruebas de efectividad y evasión

Objetivo

Evaluar cada capa individual y combinada con herramientas estandar (PDF p.15).

Que se hizo (paso a paso)

```
python3 waap/ml/bateria_fase6.py # -> matriz.csv (Tabla 2)
sqlmap vs WAAP y vs directo (search + login); ZAP baseline vs :8080
```

Resultado y lectura

sqlmap: 42x403 via WAAP y 14x403+8x500 directo ('not injectable' en ambos). ZAP: 0 FAIL, 59 PASS. Conclusion (media pagina en evidencias/fase6/analisis.md): mejor combinacion CRS+RASP con ML como senal; residuo: string-form (fix de patron) y rafagas (regla de tasa).

Analisis completo:

```
# Fase 6 - Analisis de efectividad (media pagina, PDF p.15)
```

```
## Qué combinación ofrece la mejor cobertura y por qué
```

Ninguna capa sola cubre todo; la cobertura total solo aparece al combinarlas, y cada hueco de una capa lo tapa otra (defensa en profundidad verificada):

- **Reglas CRS (PL1)**: bloquean 7/9 técnicas (clásico, encoded, UNION, XSS x2, traversal, login clásico/UNION). Fallan en lo **fragmentado** (evalúan cada parámetro por separado) y en variantes con espacios que alteran la firma (comentarios con espacios: lo pasa el RASP, no el CRS).
- **IA/ML (umbral 0.0)**: detecta 9/9 como **anómalo**, pero al costo de FPR 80% (4/5 normales marcados). Con umbral -0.05 solo 2/5 con FPR 40%. Conclusión: el modelo **no es bloqueador viable solo**; sirve como **señal** para el orquestador (permitir/bloquear/desafiar) o para revisión, no como gate duro. La **ráfaga bot** ni siquiera cruza el umbral (limitación de IsolationForest documentada en Fase 3).
- **RASP (fork)**: bloquea 4/9 en contexto de ejecución (fragmentado, comentarios con `--`, UNION y clásico de login), incluyendo las 2 evasiones del CRS. Falla en XSS/traversal (fuera de su alcance SQL) y en la forma string `' OR '1'='1'` (hueco del regex literal, con fix propuesto). Su tasa de falsos positivos es 0 en lo probado: solo dispara ante **alteración estructural real**.

Mejor combinación: CRS + RASP, con ML como **señal de apoyo**. CRS frena el grueso automatizado (sqlmap: 42x403 vía WAAP, "not injectable", sugiere tamper), RASP captura lo que evade firmas con contexto real (sqlmap directo: 14x403 + 8x500, tampoco inyectable), y ML aporta **detección** de variantes raras a cambio de **revisión humana**. ZAP baseline: 0 FAIL, 59 PASS (higiene pasiva correcta).

Residuo aceptado: string-form SQLi en search (requiere fix de patrón) y ráfagas (requieren regla de tasa explícita, no ML puro).

Matriz comparativa por capas (Fase 6, resultado final)

Tabla 2. Bateria de 9 tecnicas x 3 capas (script waap/ml/bateria_fase6.py; atribucion CRS-vs-RASP por cuerpo de respuesta).

Payload / tecnica	Reglas CRS PL1	IA/ML (umbral 0.0)	RASP
SQLi clasico	BLOQUEA 403	Detecta	EVADE en search (hueco); BLOQUEA en login
SQLi encoded	BLOQUEA 403	Detecta	EVADE en search; BLOQUEA patron OR 1=1
SQLi fragmentado q=' & x=OR 1=1	EVADE 200	No detecta	BLOQUEA 403
SQLi UNION / comentarios	BLOQUEA (compacto)	Detecta	BLOQUEA 403
XSS basico y encoded	BLOQUEA 403	Detecta	N/A (alcance SQL)
Traversal	BLOQUEA 403	Detecta	N/A (alcance SQL)
Rafaga bot	N/A	No cruza umbral	N/A

Fase 7 - Observabilidad y metricas

Objetivo

Visibilidad centralizada y numeros para tuning (PDF p.16).

Que se hizo (paso a paso)

```
python3 waap/ml/metricas_fase7.py # logs/eventos.jsonl + TPR/FPR/FNR + MTDD
```

Resultado y lectura

CRS: TPR 78% FPR 0%. ML@0.0: TPR 100% FPR 80% (senal, no gate). RASP-SQL: TPR 67% FPR 0%. MTDD ~60-150 ms por capa. Tuning: CRS en PL1; ML solo como senal + regla hibrida de tasa; RASP con patron ampliado. Detalles:

capa		TPR	FPR	FNR	
reglas CRS PL1		78%	0%	22%	(TP=7 FN=2 FP=0 TN=5)
IA/ML umbral 0.0		100%	80%	0%	(TP=9 FN=0 FP=4 TN=1)
RASP fork (SQL)		67%	0%	33%	(TP=4 FN=2 FP=0 TN=5)

MTDD simulado (tiempo-a-veredicto por capa):

- CRS/sqli-clasico: 144.0 ms
- CRS/xss: 124.0 ms
- RASP/sqli-login: 61.5 ms

OK logs/eventos.jsonl (8 eventos)

Cuestionario de analisis final (PDF Seccion 8)

Q1. Motor de reglas insuficiente pero IA/ML si detecta.

El SQLi con comentarios y espacios ('/ **/OR/**/1=1--): la firma exacta del CRS no lo casa (403 solo del RASP por el '--'), mientras el ML lo marca anomalo por longitud+entropia atipicas. La firma busca cadenas conocidas; el modelo mide distancia estadistica.

Q2. Falso positivo del ML sobre trafico legitimo.

El login valido y el home ('/') se marcaron anomalos: el login por body_length alto del JSON de credenciales y home por url_length=1 (extremo de la distribucion). Feature responsable principal: longitudes fuera del centro aprendido + entropia.

Q3. Por que el RASP detecto lo que reglas y ML no.

Porque opera sobre la consulta final concatenada (contexto de ejecucion), no sobre la peticion HTTP. El fragmentado q=' & x=OR 1=1 es inocuo por partes (evade CRS y es estadisticamente normal para el ML) pero al concatenarse forma ' OR 1=1--', que el guard ve y bloquea con 403.

Q4. WAAP centralizado vs RASP en 15 microservicios.

WAAP centralizado como base (un solo punto de tuning, cubre OWASP Top 10 generico) + RASP solo en servicios criticos con datos sensibles (auth, pagos), por el costo operativo de mantener 15 agentes (versionado por lenguaje, overhead de despliegue, logs).

Q5. Que etapa detuvo la vulnerabilidad inyectada.

SAST/Semgrep detuvo el eval() inyectado (code-string-concat, blocking). Sin esa etapa, el codigo habria llegado a la imagen y solo ZAP/Trivy tardios (o produccion) lo habrian visto, con costo mayor.

Q6. Mapeo MITRE ATT&CK / API Top 10.

T1190 (Exploit Public-Facing Application): SQLi en /rest/user/login y XSS en /rest/products/search. API BOLA/IDOR: acceso a /rest/basket/:id ajeno.

Q7. Mejora contra falsos positivos del ML sin perder deteccion.

Regla hibrida: score ML + tasa (req_per_minute con umbral duro) + whitelist por ruta (p. ej. login/home con umbral propio). La tasa captura rafagas que el IsolationForest no ordena; el whitelist baja el FPR sin tocar la deteccion de ataques.

Q8. Riesgo de reentrenar con trafico de produccion sin validar.

Envenenamiento: ataques de bajo volumen se vuelven 'normales' (lo vivimos al reves: el dataset rapido normalizo rpm=200). Se requiere curaduria + ventana validada + comparacion contra baseline antes de promover el modelo.

Anexo A - Paso a paso reproducible (Fases 0-7)

```
# 0. Requisitos: Ubuntu 22.04, Docker+Compose, Python 3.10, Node 22, LibreOffice.
#   sudo usermod -aG docker $USER (re-login).
git clone git@github.com:Jhojan98/taller-waap.git && cd taller-waap
git checkout lab/fases-0-4 # o main tras el merge
# FASE 0: cp .env.example .env; docker compose pull/up; curl :8080
# FASE 1: version, search?q=apple, /#/score-board
# FASE 2: SQLi clasico -> 403; fragmentado -> 200; logs | grep 942100|949110
# FASE 3: pip install sklearn pandas numpy joblib; GEN_N=120 PACE=humano
gen_traffic;
#   extract_features; train_model; evaluate
# FASE 4: fork juice-shop (rama rasp-lab) + lib/raspGuard.ts + hooks;
#   PORT=3001 npm start; docker build -t juice-shop:rasp-lab; compose + verificar
# FASE 5: semgrep; pip-audit; docker build ./app; trivy; checkov; deploy script
# FASE 6: bateria_fase6.py; sqlmap (search+login); zap-baseline.py
# FASE 7: metricas_fase7.py
# INFORME: python3 build_informe_v2.py; soffice --headless --convert-to pdf ...docx
```

Anexo B - Salidas de terminal (Fases 0-2)

```
===== docker compose ps =====
NAME                                IMAGE                                COMMAND
SERVICE    CREATED          STATUS          PORTS
taller-waap-juice-shop-1    bkimminich/juice-shop:latest    "/nodejs/bin/node /jâ€¢"
juice-shop    32 minutes ago  Up 32 minutes    0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp
taller-waap-waap-proxy-1    owasp/modsecurity-crs:nginx    "/docker-entrypoint.â€¢"
waap-proxy    32 minutes ago  Up 32 minutes (healthy)    0.0.0.0:8080->8080/tcp,
[::]:8080->8080/tcp

===== curl cÃ³digos =====
HOME via WAAP 8080: 200
search normal: 200
sqli clasico: 403
evasion fragmentada: 200

===== WAAP env =====
PARANOIA=1
BACKEND=http://juice-shop:3000
MODSEC_DISABLE_BACKEND_COMPRESSION=off
BLOCKING_PARANOIA=1

===== regla CRS (bloqueo) =====
34 "ruleId":"949110"
34 "message":"Inbound Anomaly Score Exceeded (Total Score: 5)"
33 "message":"Request content type is not allowed by policy"
1 "ruleId":"942100"
1 "message":"SQL Injection Attack Detected via libinjection"
```